

LO5: Reviews, Inspections and Automated Tests

LO5.1: Although extensive automated testing was carried out, code review was used as an additional quality assurance technique to reduce residual risk, particularly for safety-critical and cost-sensitive logic. Reviews focused on complex parts of the system where faults could have significant downstream effects, notably the path-finding and cost calculation logic in `IlpDroneService` and the geometric safety checks in `geometry`. A checklist was applied, including:

- Correct enforcement of movement and safety rules, such as fixed step size, valid angles, and no crossing no fly zones.
- Correct handling of boundary conditions and loop termination in path generation.
- Consistent use of validation logic for coordinates, angles, and requests.
- Correct cost and move counting, with no unintended side effects.
- Clarity of control flow in safety-critical code.

This review process identified a subtle fault where an extra hover step was added at each delivery point, which meant that although the drone reached the right locations and passed the functional tests, this affected the total move count and delivery cost over time. This was not originally identified in the implemented tests, but was revealed in review and marking.

The fault was then fixed in the implementation, and in future a test should be added to check for this issue, as this mistake highlights a clear gap in testing. This test should check for the final number of moves and cost against expected, and should reduce the risk for similar errors in future. This is especially important for errors that are not directly visible through functional behaviour alone, and this scenario demonstrates how code review can reveal gaps in test coverage.

LO5.2: A continuous integration (CI) pipeline was implemented using GitHub Actions, following the recommended structure. The pipeline is defined in a workflow that triggers automatically for every push and pull request. The CI pipeline performs the following steps:

1. Checks out the repository
2. Sets up a Java 21 environment
3. Caches Maven dependencies to improve build speed
4. Executes maven package, which compiles the system and runs the full automated test suite.

This pipeline is appropriate for the drone delivery system because it ensures that every code change is automatically validated against the full set of functional, integration, safety and performance tests before being merged. In a bigger project, this pipeline could be extended with security checks, consistency checks or deployment, but for this coursework it allows for fast and reliable feedback and catches requirement violations before they enter the repo.

LO5.3: Testing was automated using JUnit and Spring Boot's testing framework, and fully integrated into the CI pipeline. The automated test suite includes unit tests for `geometry` and validation logic, integration tests for interactions between controllers, validation and `geometry` services and system and performance tests for `calcDeliveryPath`.

All tests run automatically as part of the maven build. External dependencies and endpoints are mocked using RestTemplate stubs to ensure tests run quickly and are deterministic, making it suitable for CI execution. Fast, repeatable tests are prioritised for continuous integration.

LO5.4: The CI pipeline functions as intended, shown in the below screenshot.

6 workflow runs

✓

more fixed tests

Java CI with Maven #6: Commit [468afb1](#) pushed by [ebleele](#)

main

📅

Today at 12:16 AM

🕒

33s

...

✗

more tests

Java CI with Maven #5: Commit [304964b](#) pushed by [ebleele](#)

main

📅

Jan 19, 11:10 PM GMT

🕒

21s

...

✓

performance tests

Java CI with Maven #4: Commit [35c21ee](#) pushed by [ebleele](#)

main

📅

Jan 16, 12:23 PM GMT

🕒

41s

...

✗

Merge remote-tracking branch 'origin/main'

Java CI with Maven #3: Commit [b65c803](#) pushed by [ebleele](#)

main

📅

Jan 15, 9:25 PM GMT

🕒

21s

...

✓

Update maven.yml

Java CI with Maven #2: Commit [1223b28](#) pushed by [ebleele](#)

main

📅

Jan 13, 4:12 PM GMT

🕒

41s

...

✗

Create maven.yml

Java CI with Maven #1: Commit [38f0eb0](#) pushed by [ebleele](#)

main

📅

Jan 13, 4:10 PM GMT

🕒

43s

...

The actions history shows multiple workflow runs, including earlier failed builds caused by configuration issues and newly introduced test failures, followed by successful builds once the issues were fixed. This provides concrete evidence that the pipeline detects faults and prevents regressions from being silently introduced. The pipeline is effective at identifying multiple issues including compilation and dependency errors, failures in unit, integration, safety and performance tests, and violations of high priority requirements.

However, the pipeline is limited in the way it cannot detect issues that fall outside the scope of the test suite. Within the constraints of the project, the CI pipeline provides a strong automated safety net and demonstrates a good alignment between testing strategy, automation and continuous integration practice.